DMS – LAB 2 – INTEGRITY CONSTRAINTS IN SQL

Name: Sudhiksha Shanbhag

Registration Number: 240958004

Roll number: 5

Branch: Computer Science and Finance Technology

PART 1: Consider the following schema: Employee (EmpNo, EmpName, Gender, Salary, Address, DNo) Department (DeptNo, DeptName, Location)

Question 1: Create Employee table with following constraints:

• Make EmpNo as Primary key.

• Do not allow EmpName, Gender, Salary and Address to have null values.

• Allow Gender to have one of the two values: 'M', 'F'.

Code:

```
CREATE TABLE EMPLOYEE (
  2  EMPNO NUMBER(5) PRIMARY KEY,
  3  EMPNAME VARCHAR(50) NOT NULL,
  4  GENDER CHAR(1) CHECK (GENDER IN ('M', 'F')),
  5  SALARY NUMBER(10) NOT NULL,
  6  ADDRESS VARCHAR(100) NOT NULL,
  7  DNO NUMBER(5));
```

Output:

```
Table created.
```

```
SQL> DESC EMPLOYEE;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 EMPNO                                     NOT NULL NUMBER(5)
 EMPNAME                                   NOT NULL VARCHAR2(50)
 GENDER                                             CHAR(1)
 SALARY                                    NOT NULL NUMBER(10)
 ADDRESS                                   NOT NULL VARCHAR2(100)
 DNO                                                NUMBER(5)
```

Question 2: Create Department table with following: Make DeptNo as Primary key

• Make DeptName as candidate key

Code:

```
CREATE TABLE DEPARTMENT (
 2  DEPTNO NUMBER(5) PRIMARY KEY,
 3  DEPTNAME VARCHAR(50) UNIQUE NOT NULL,
 4  LOCATION VARCHAR(50));
```

Output:

```
Table created.
```

```
SQL> DESC DEPARTMENT;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 DEPTNO                                    NOT NULL NUMBER(5)
 DEPTNAME                                  NOT NULL VARCHAR2(50)
 LOCATION                                           VARCHAR2(50)
```

Question 3: Make DNo of Employee as foreign key which refers to DeptNo of Department.

Code:

```
ALTER TABLE EMPLOYEE ADD CONSTRAINT FK_DEPT FOREIGN KEY (DNO)
REFERENCES DEPARTMENT (DEPTNO);
```

Output:

```
Table altered.
```

Question 4: Insert few tuples into Employee and Department which satisfies the above constraints.

Code:

```
INSERT INTO DEPARTMENT VALUES(10, 'HR', 'MUMBAI');
```

```
1 row created.
```

INSERT INTO DEPARTMENT VALUES(20, 'IT', 'DEHLI');

```
1 row created.
```

INSERT INTO DEPARTMENT VALUES(30, 'SALES', 'BANGALORE');

```
1 row created.
```

INSERT INTO EMPLOYEE VALUES(101, 'ALICE', 'F', 50000, 'DEHLI', 20);

```
1 row created.
```

INSERT INTO EMPLOYEE VALUES(102, 'BOB', 'M', 45000, 'MUMBAI', 10);

```
1 row created.
```

INSERT INTO EMPLOYEE VALUES(103, 'CLARA', 'F', 55000, 'BANGALORE', 30);

```
1 row created.
```

Output:

```
SQL> SELECT * FROM DEPARTMENT;

    DEPTNO DEPTNAME       LOCATION
---------- -------------- --------------------------------------------------
        10 HR             MUMBAI
        20 IT             DEHLI
        30 SALES          BANGALORE
```

```
SQL> SELECT * FROM EMPLOYEE;

EMPNO EMPNAME         GENDER  SALARY ADDRESS                    DNO
----- --------------- ------ ------- -------------------- ----------
  101 ALICE           F        50000 DEHLI                        20
  102 BOB             M        45000 MUMBAI                       10
  103 CLARA           F        55000 BANGALORE                    30
```

Question 5: Try to insert few tuples into Employee and Department which violates some of the above constraints.

Code:

INSERT INTO EMPLOYEE VALUES (104, NULL, 'M', 40000, 'KOLKATA', 10);

INSERT INTO EMPLOYEE VALUES (105, 'DAVID', 'X', 40000, 'PUNE', 20);

INSERT INTO EMPLOYEE VALUES (101, 'EVE', 'F', 40000, 'GOA', 10);

INSERT INTO EMPLOYEE VALUES (106, 'FRANK', 'M', 38000, 'JAIPUR', 99);

Output:

```
SQL> INSERT INTO EMPLOYEE VALUES (104, NULL, 'M', 40000, 'KOLKATA', 10);
INSERT INTO EMPLOYEE VALUES (104, NULL, 'M', 40000, 'KOLKATA', 10)
                                 *
ERROR at line 1:
ORA-01400: cannot insert NULL into ("SYSTEM"."EMPLOYEE"."EMPNAME")
```

```
SQL> INSERT INTO EMPLOYEE VALUES (105, 'DAVID', 'X', 40000, 'PUNE', 20);
INSERT INTO EMPLOYEE VALUES (105, 'DAVID', 'X', 40000, 'PUNE', 20)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.SYS_C007559) violated
```

```
SQL> INSERT INTO EMPLOYEE VALUES (101, 'EVE', 'F', 40000, 'GOA', 10);
INSERT INTO EMPLOYEE VALUES (101, 'EVE', 'F', 40000, 'GOA', 10)
*
ERROR at line 1:
ORA-00001: unique constraint (SYSTEM.SYS_C007560) violated
```

```
SQL> INSERT INTO EMPLOYEE VALUES (106, 'FRANK', 'M', 38000, 'JAIPUR', 99);
INSERT INTO EMPLOYEE VALUES (106, 'FRANK', 'M', 38000, 'JAIPUR', 99)
*
ERROR at line 1:
ORA-02291: integrity constraint (SYSTEM.FK_DEPT) violated - parent key not found
```

Question 6: Try to modify/delete a tuple which violates a constraint. (Ex: Drop a department tuple which has one or more employees)

Code:

DELETE FROM DEPARTMENT WHERE DEPTNO = 10;

Output:

```
SQL> DELETE FROM DEPARTMENT WHERE DEPTNO = 10;
DELETE FROM DEPARTMENT WHERE DEPTNO = 10
*
ERROR at line 1:
ORA-02292: integrity constraint (SYSTEM.FK_DEPT) violated - child record found
```

Question 7: Modify the foreign key constraint of Employee table such that whenever a depart- ment tuple is deleted, the employees belonging to that department will also be deleted.

Code:

```
ALTER TABLE EMPLOYEE DROP CONSTRAINT FK_DEPT;

ALTER TABLE EMPLOYEE ADD CONSTRAINT FK_DEPT FOREIGN KEY (DNO)
REFERENCES DEPARTMENT (DEPTNO) ON DELETE CASCADE;

DELETE FROM DEPARTMENT WHERE DEPTNO = 30;
```

Output:

```
Table altered.
```

```
Table altered.
```

```
1 row deleted.
```

```
SQL> SELECT * FROM EMPLOYEE;

EMPNO EMPNAME         GENDER  SALARY ADDRESS                    DNO
----- --------------- ------ ------- -------------------- ----------
  101 ALICE           F        50000 DEHLI                        20
  102 BOB             M        45000 MUMBAI                       10
```

Question 8: Create a named constraint to set the default salary to 10000 and test the constraint by inserting a new record.

Code:

ALTER TABLE EMPLOYEE MODIFY SALARY DEFAULT 10000;

Output:

```
Table altered.
```

```
SQL> INSERT INTO EMPLOYEE(EMPNO, EMPNAME, GENDER, ADDRESS, DNO) VALUES (108, 'HENRY', 'M',
 'CHENNAI', 20);

1 row created.
```

Shows that theres already a default in place as salary can't be null too, it has to have some value – default, for the above query/command to execute.

PART 2: Account (account_number, branch_name, balance) 15 branch (branch_name, branch_city, assets) customer (customer_name customer_street, customer_city) loan (loan_number, branch_name, amount) depositor (customer_name, account_number) borrower (customer_name, loan_number) Note: Use BankDB.sql to create and populate the Database.

Question 1: List all branch names and their assets.

Code:

SELECT BRANCH_NAME, ASSETS FROM BRANCH;

Output:

```
BRANCH_NAME                      ASSETS
-------------------------------- ----------
PERRYRIDGE                       400000
DOWNTOWN                         900000
MIANUS                           700000
BROOKLYN                         500000
```

Question 2: List all accounts of Brooklyn branch.

Code:

SELECT * FROM ACCOUNT WHERE BRANCH_NAME = 'BROOKLYN';

Output:

ACCOUNT_NUMBER	BRANCH_NAME	BALANCE
A-303	BROOKLYN	1200
A-304	BROOKLYN	850

Question 3: List all loans with amount > 1000.

Code:

SELECT * FROM LOAN WHERE AMOUNT > 1000;

Output:

LOAN_NUMBER	BRANCH_NAME	AMOUNT
L-14	PERRYRIDGE	1500

Question 4: List all accounts of Perryridge branch with balance < 1000.

Code:

SELECT * FROM ACCOUNT WHERE BRANCH_NAME = 'PERRYRIDGE' AND BALANCE < 1000;

Output:

ACCOUNT_NUMBER	BRANCH_NAME	BALANCE
A-102	PERRYRIDGE	400

Question 5: List Numbers of accounts with balances between 700 and 900.

Code:

SELECT ACCOUNT_NUMBER FROM ACCOUNT WHERE BALANCE BETWEEN 700 AND 900;

Output:

ACCOUNT_NUMBER
A-215
A-304

Question 6: Change the assests of Perryridge branch to 340000000.

Code:

```
UPDATE BRANCH SET ASSETS = 340000000 WHERE BRANCH_NAME = 'PERRYRIDGE';
```

Output:

```
1 row updated.
```

```
SQL> SELECT * FROM BRANCH;

BRANCH_NAME                    BRANCH_CITY            ASSETS
------------------------------ -------------------- ----------
PERRYRIDGE                     HORSENECK            340000000
DOWNTOWN                       BROOKLYN                900000
MIANUS                         HORSENECK               700000
BROOKLYN                       BROOKLYN                500000
```

Question 7: Transfer the accounts and loans of Perryridge branch to Downtown branch.

Code:

```
UPDATE ACCOUNT SET BRANCH_NAME = 'DOWNTOWN' WHERE BRANCH_NAME =
'PERRYRIDGE';

UPDATE LOAN SET BRANCH_NAME = 'DOWNTOWN' WHERE BRANCH_NAME =
'PERRYRIDGE';
```

Output:

```
1 row updated.
```

```
SQL> SELECT * FROM ACCOUNT;

ACCOUNT_NUMBER       BRANCH_NAME                      BALANCE
-------------------- ------------------------------ ----------
A-101                DOWNTOWN                            500
A-102                DOWNTOWN                            400
A-215                MIANUS                              750
A-303                BROOKLYN                           1200
A-304                BROOKLYN                            850
```

```
1 row updated.
```

```
SQL> SELECT * FROM LOAN;

LOAN_NUMBER     BRANCH_NAME                      AMOUNT
--------------- ------------------------------ ----------
L-11            DOWNTOWN                          900
L-14            DOWNTOWN                         1500
```

Question 8: Transfer Rs. 100 from account A-101 to A-215.

Code:

UPDATE ACCOUNT SET BALANCE = BALANCE - 100 WHERE ACCOUNT_NUMBER = 'A-101';

UPDATE ACCOUNT SET BALANCE = BALANCE + 100 WHERE ACCOUNT_NUMBER = 'A-215';

Output:

```
1 row updated.
```

```
1 row updated.
```

```
SQL> SELECT * FROM ACCOUNT;

ACCOUNT_NUMBER      BRANCH_NAME                      BALANCE
------------------- ------------------------------ ----------
A-101               DOWNTOWN                            500
A-102               DOWNTOWN                            400
A-215               MIANUS                              750
A-303               BROOKLYN                           1200
A-304               BROOKLYN                            850
```

```
SQL> SELECT * FROM ACCOUNT;

ACCOUNT_NUMBER      BRANCH_NAME                       BALANCE
------------------- ------------------------------ ----------
A-101               DOWNTOWN                              400
A-102               DOWNTOWN                              400
A-215               MIANUS                                850
A-303               BROOKLYN                             1200
A-304               BROOKLYN                              850
```

Question 9: Delete the branch Perryridge.

Code:

DELETE FROM BRANCH WHERE BRANCH_NAME = 'PERRYRIDGE';

Output:

```
1 row deleted.
```

```
SQL> SELECT * FROM BRANCH;

BRANCH_NAME                    BRANCH_CITY              ASSETS
------------------------------ -------------------- ----------
DOWNTOWN                       BROOKLYN                900000
MIANUS                         HORSENECK               700000
BROOKLYN                       BROOKLYN                500000
```

Question 10: Waive off all the loans with amount < 1000.

Code:

DELETE FROM BORROWER WHERE LOAN_NUMBER IN (SELECT LOAN_NUMBER FROM LOAN WHERE AMOUNT < 1000);

DELETE FROM LOAN WHERE AMOUNT < 1000;

Output:

```
SQL> DELETE FROM BORROWER WHERE LOAN_NUMBER IN ( SELECT LOAN_NUMBER FROM LOAN WHERE AMOUNT
 < 1000 );

1 row deleted.

SQL> DELETE FROM LOAN WHERE AMOUNT < 1000;

1 row deleted.
```

Question 11: Delete the accounts and loans of Downtown branch.

Code:

```
DELETE FROM DEPOSITOR WHERE ACCOUNT_NUMBER IN ( SELECT
ACCOUNT_NUMBER FROM ACCOUNT WHERE BRANCH_NAME = 'DOWNTOWN');

DELETE FROM ACCOUNT WHERE BRANCH_NAME = 'DOWNTOWN';

DELETE FROM BORROWER WHERE LOAN_NUMBER IN ( SELECT LOAN_NUMBER FROM
LOAN WHERE BRANCH_NAME = 'DOWNTOWN');

DELETE FROM  LOAN WHERE BRANCH_NAME = 'DOWNTOWN';
```

Output:

```
1 row deleted.
```

```
2 rows deleted.
```

```
1 row deleted.
```

```
1 row deleted.
```

```
SQL> SELECT * FROM ACCOUNT;

ACCOUNT_NUMBER       BRANCH_NAME                       BALANCE
-------------------- ------------------------------ ----------
A-215                MIANUS                                850
A-303                BROOKLYN                             1200
A-304                BROOKLYN                              850
```

```
SQL> SELECT * FROM LOAN;

no rows selected
```

Question 12: Add a column phoneNo to customer table.

Code:

```
ALTER TABLE CUSTOMER ADD PHONENO VARCHAR(15);
```

Output:

```
Table altered.
```

```
SQL> DESC CUSTOMER;
 Name                                                                    Null?    Type
 ----------------------------------------------------------------------- -------- ------------
-----------------------------
 CUSTOMER_NAME                                                           NOT NULL VARCHAR2(50)
 CUSTOMER_STREET                                                                  VARCHAR2(50)
 CUSTOMER_CITY                                                                    VARCHAR2(30)
 PHONENO                                                                          VARCHAR2(15)
```

Question 13: Change the size of the branch_city to varchar(20).

Code:

```
ALTER TABLE BRANCH MODIFY BRANCH_CITY VARCHAR(20);
```

Output:

```
Table altered.
```

```
SQL> DESC BRANCH;
 Name                                                                    Null?    Type
 ----------------------------------------------------------------------- -------- ------------
-----------------------------
 BRANCH_NAME                                                             NOT NULL VARCHAR2(30)
 BRANCH_CITY                                                                      VARCHAR2(20)
 ASSETS                                                                           NUMBER(15,2)
```

Question 14: Drop the column phoneNo from customer table.

Code:

```
ALTER TABLE CUSTOMER DROP COLUMN PHONENO;
```

Output:

```
Table altered.
```

```
SQL> DESC CUSTOMER;
 Name                                                      Null?    Type
 ----------------------------------------------------------------- -------- ------------
------------------------------
 CUSTOMER_NAME                                             NOT NULL VARCHAR2(50)
 CUSTOMER_STREET                                                    VARCHAR2(50)
 CUSTOMER_CITY                                                      VARCHAR2(30)
```

Question 15: For all customers who have a loan from the bank, find their names, loan numbers, and loan amount.

Code:

SELECT BORROWER.CUSTOMER_NAME, LOAN.LOAN_NUMBER, LOAN.AMOUNT FROM BORROWER JOIN LOAN ON BORROWER.LOAN_NUMBER = LOAN.LOAN_NUMBER;

Output:

CUSTOMER_NAME	LOAN_NUMBER	AMOUNT
John Doe	L-101	1500

Question 16: Find the customer names, loan numbers, and loan amounts, for all loans at the Perryridge branch.

Code:

SELECT BORROWER.CUSTOMER_NAME, LOAN.LOAN_NUMBER, LOAN.AMOUNT FROM BORROWER JOIN LOAN ON BORROWER.LOAN_NUMBER = LOAN.LOAN_NUMBER;

Output:

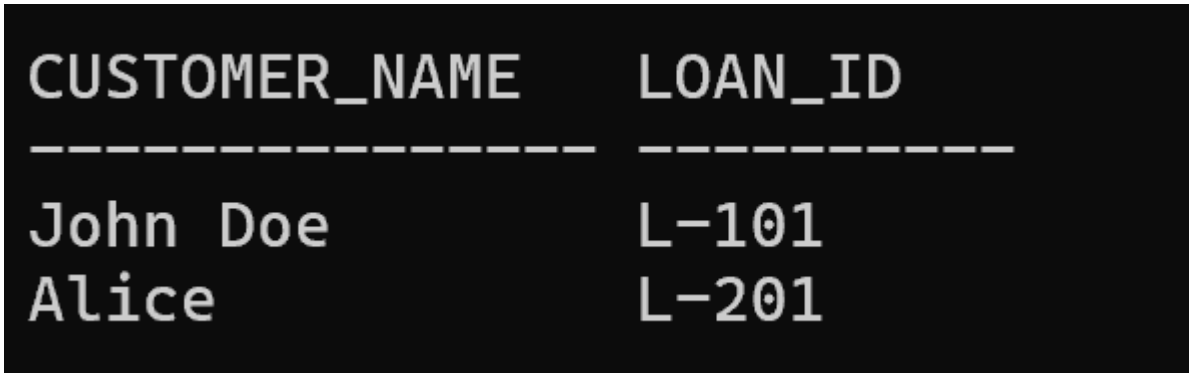
CUSTOMER_NAME	LOAN_NUMBER	AMOUNT
John Doe	L-101	1500
Alice	L-201	2000

Question 17: For all customers who have a loan from the bank, find their names and loan numbers with the attribute loan_number replaced by loan_id.

Code:

SELECT BORROWER.CUSTOMER_NAME, LOAN.LOAN_NUMBER AS LOAN_ID FROM BORROWER JOIN LOAN ON BORROWER.LOAN_NUMBER = LOAN.LOAN_NUMBER;

Output:



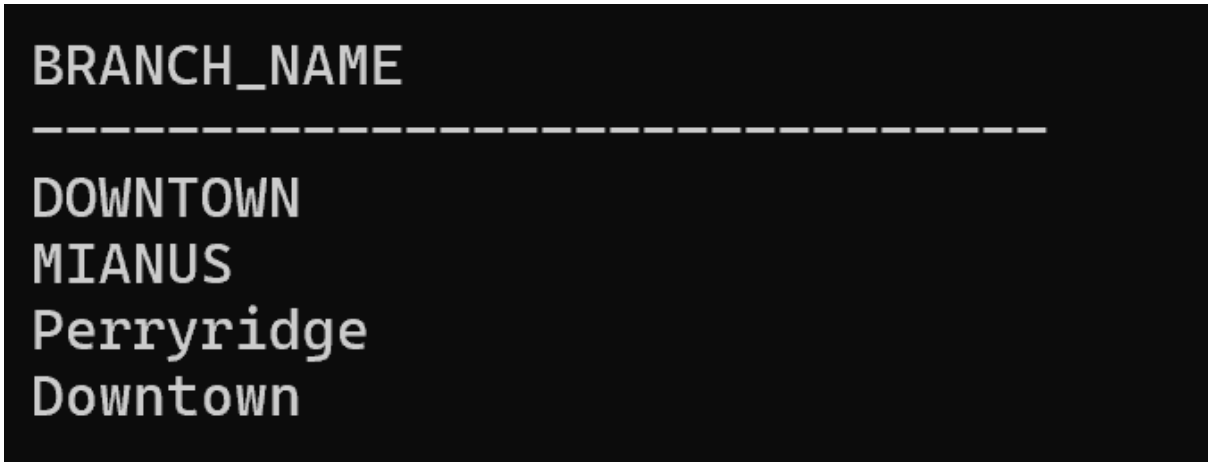
CUSTOMER_NAME	LOAN_ID
John Doe	L-101
Alice	L-201

Question 18: Find the names of all branches that have assets greater than atleast one branch located in Brooklyn.

Code:

```
SELECT BRANCH_NAME FROM BRANCH WHERE ASSETS > ANY ( SELECT ASSETS
FROM BRANCH WHERE BRANCH_CITY = 'BROOKLYN');
```

Output:



BRANCH_NAME
DOWNTOWN
MIANUS
Perryridge
Downtown

Question 19: Find the names of all customers whose street address includes the substring ‘Main’.

Code:

```
SELECT CUSTOMER_NAME FROM CUSTOMER WHERE CUSTOMER_STREET LIKE
'%MAIN%';
```

Output:

```
CUSTOMER_NAME
---------------
JONES
HAYES
BAKER
```